Project Overview

Project Title:

Smart Community Health Monitoring & Early Warning System for Water-Borne Diseases

Description:

A real-time, AI-powered community health monitoring platform to detect, track, and predict water-borne disease outbreaks (cholera, typhoid, dysentery) using IoT water quality sensors, patient symptom reporting, machine learning models, and geo-spatial analytics. The system provides early warnings to health authorities and communities before outbreaks escalate.

Technology Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Spring Boot (Java)
AI / ML Module	Python (Scikit-learn / TensorFlow / Pandas)
IoT Integration	MQTT Protocol + Sensor APIs
Database	PostgreSQL + InfluxDB (time-series sensor data)
Geo-Analytics	Google Maps API + D3.js heatmaps
Notifications	Firebase Push + Twilio SMS + Email
Security	JWT + Spring Security + HTTPS
Deployment	Docker + AWS EC2 / Render

DAY
01

Date: Jun-01

Project Title Finalization

Expected Deliverable

Approved Project Title

Brainstorming & Selection

The team evaluated multiple public health and IoT-based project ideas. Water-borne disease outbreaks are a critical, recurring public health challenge in India, making this topic both impactful and technically rich.

Why This Topic?

Reason	Explanation
Real-World Need	Water-borne diseases kill ~1.2 million people/year globally; early warning saves lives
Technology Depth	Combines IoT + AI/ML + Geo-Spatial + Spring Boot + React -- covers full stack
Government Relevance	Aligns with National Water Quality Monitoring Mission and Smart Cities initiative
Data Richness	Sensor streams + patient data = complex, interesting ML problem
Academic Value	Covers all 10 modules, 3NF DB, REST APIs, and ML pipelines

Title Finalization Steps

- Generated 12 project ideas in team brainstorm session
- Shortlisted 3: Smart Water Quality Monitor, Hospital Appointment AI, Community Disease Alert System
- Voted and finalized: Smart Community Health Monitoring & Early Warning System for Water-Borne Diseases
- Faculty reviewed and approved the title with positive feedback

DAY
02

Date: Jun-02

Requirement Gathering

Expected Deliverable

Problem Statement

Problem Statement

Problem Statement:

Communities, especially in rural and semi-urban areas, lack a real-time system to detect and respond to water-borne disease outbreaks. Water quality data from municipal sources is not integrated with health records, making it impossible to correlate contamination events with disease spikes. By the time an outbreak is identified through manual reporting, hundreds of people are already affected. There is an urgent need for an AI-driven, sensor-integrated, geo-aware early warning platform.

Stakeholder Analysis

Stakeholder	Requirements
Community Members	Report symptoms, receive outbreak warnings, view safe water sources on map
Health Officers / Doctors	Dashboard to track patient symptom clusters, trigger alerts, manage cases
Water Authority / Municipality	Monitor IoT sensor readings, get contamination alerts, manage water quality
Government / Public Health Dept	Analytics: outbreak trends, risk zones, resource allocation, disease forecasting
System Admin	Manage users, sensors, zones, AI models, notification channels
NGOs / Aid Workers	Access outbreak maps, coordinate relief efforts in affected zones

Functional Requirements

- Real-time IoT water quality sensor data ingestion (pH, turbidity, TDS, coliform)
- Community symptom reporting through mobile-friendly web app
- AI/ML-based outbreak prediction from symptom + sensor data patterns
- Geo-spatial heatmap showing disease risk zones by locality
- Automated early warning alerts to health officers and community
- Case management dashboard for health officers
- Water quality trend analytics with historical comparison

DAY
03

Date: Jun-03

Objective Definition

Expected Deliverable

Project Objectives

Project Objectives

Obj #	Objective Description
OBJ-01	Build a real-time IoT sensor dashboard displaying water quality parameters (pH, TDS, turbidity, coliform levels)
OBJ-02	Develop a community symptom reporting module for residents to log health symptoms with geo-location
OBJ-03	Implement an AI/ML early warning model that predicts disease outbreak probability by zone
OBJ-04	Create geo-spatial heatmaps showing disease risk levels and water quality across areas
OBJ-05	Build an automated multi-channel alert system (SMS, push, email) for health authorities and citizens
OBJ-06	Develop a health officer case management dashboard to track, escalate, and resolve outbreak events
OBJ-07	Implement secure REST APIs with JWT authentication for all system components
OBJ-08	Enable historical trend analysis and predictive reporting for government health departments

SMART Goals

Criterion	Description
Specific	Build a water-borne disease early warning system with IoT + AI + Geo-mapping in 30 days
Measurable	All 8 objectives delivered with working demos and test reports
Achievable	Java/React/Python stack -- all team members have required skills
Relevant	Directly addresses recurring public health crises in water-scarce regions
Time-Bound	Complete full project by June 30, 2025 with PPT and report

DAY
04

Date: Jun-04

User & Module Identification

Expected Deliverable

Module List

User Roles

User Role	Description	Access Level
Community Member	Resident who reports symptoms and views health alerts	Basic
Health Officer	Doctor/PHC officer who monitors cases and triggers responses	Medium
Water Authority	Municipality worker who manages sensor data and water zones	Medium
Government Official	State/district health department -- analytics and reporting	Read-Only High
System Admin	Full system access: users, sensors, AI config, zones	Full
NGO Worker	Reads outbreak maps, coordinates in affected zones	Limited

Module List

Module #	Module Name	Description
M-01	User Authentication	Registration, Login, JWT, Role-based access control
M-02	Symptom Reporting	Community symptom submission with GPS, disease category, severity
M-03	IoT Sensor Dashboard	Real-time sensor data display (pH, TDS, turbidity, coliform)
M-04	AI Outbreak Prediction	ML model scoring outbreak risk per zone from sensor + symptom data
M-05	Geo-Spatial Heatmap	Interactive map with color-coded disease and water quality zones
M-06	Alert & Notification	Multi-channel automated alerts to health officers and communities
M-07	Health Officer Dashboard	Case list, patient symptom clusters, zone risk status, escalation
M-08	Water Quality Management	Sensor configuration, threshold setting, contamination logging
M-09	Admin Panel	User management, sensor registry, AI model config, system settings
M-10	Analytics & Reporting	Trend charts, outbreak history, predictive forecasts, export to PDF

DAY
05

Date: Jun-05

Use Case Diagram Preparation

Expected Deliverable

CRUD APIs

Use Cases by Actor

Actor	Use Case
Community Member	Register and Login
Community Member	Submit symptom report with location
Community Member	View disease risk level for my area
Community Member	Receive outbreak warning alert
Community Member	View nearby safe water sources
Health Officer	Login to dashboard
Health Officer	View symptom cluster map
Health Officer	Open and manage a disease case
Health Officer	Trigger manual outbreak alert
Health Officer	Generate incident report
Water Authority	Monitor live sensor readings
Water Authority	Set water quality thresholds
Water Authority	Log contamination event
Govt Official	View state-level outbreak analytics
Govt Official	Export monthly health reports
Admin	Manage users and roles
Admin	Register and configure IoT sensors
Admin	Update AI model parameters
System (AI)	Auto-predict outbreak risk every hour
System (Alert Engine)	Auto-send alerts when thresholds exceeded

CRUD API Matrix

Entity	Create	Read	Update	Delete
Users	POST /api/users/register	GET /api/users/{id}	PUT /api/users/{id}	DELETE /api/users/{id}
Symptom Reports	POST /api/symptoms	GET /api/symptoms	PUT /api/symptoms/{id}	DELETE /api/symptoms/{id}

Sensor Data	POST /api/sensors/data	GET /api/sensors/{id}/readings	-- (auto-logged)	-- (retain policy)
Disease Cases	POST /api/cases	GET /api/cases	PUT /api/cases/{id}	DELETE /api/cases/{id}
Zones	POST /api/zones	GET /api/zones	PUT /api/zones/{id}	DELETE /api/zones/{id}
Alerts	POST /api/alerts	GET /api/alerts	PUT /api/alerts/{id}	--
Sensors	POST /api/sensors	GET /api/sensors	PUT /api/sensors/{id}	DELETE /api/sensors/{id}

DAY
06

Date: Jun-06

Database Requirement Analysis

Expected Deliverable

Table List

Database Tables

Table Name	Purpose	Key Columns
users	All user accounts with roles	user_id, name, email, password_hash, role, area_id
community_areas	Geographic areas/zones of community	area_id, area_name, district, state, coordinates_json, risk_level
iot_sensors	Registered IoT water quality sensors	sensor_id, sensor_name, area_id, location_lat, location_lng, status
sensor_readings	Time-series water quality readings from IoT	reading_id, sensor_id, ph, tds, turbidity, coliform, recorded_at
symptom_reports	Community disease symptom submissions	report_id, user_id, area_id, symptoms_json, severity, reported_at
disease_cases	Health officer managed outbreak cases	case_id, disease_type, area_id, reported_by, status, opened_at
outbreak_predictions	AI model predictions per zone per time	prediction_id, area_id, risk_score, risk_level, model_version, predicted_at
alerts	System-generated alerts sent to users	alert_id, type, message, area_id, severity, sent_at, channel
alert_recipients	Alert delivery tracking per user	recipient_id, alert_id, user_id, delivered_at, read_status
water_contamination_logs	Logged contamination events	log_id, sensor_id, contaminant_type, severity, logged_at, resolved_at

Database Design Decisions

- PostgreSQL: Relational data -- users, cases, zones, sensors, alerts
- InfluxDB: Time-series sensor readings (optimized for high-frequency IoT data)
- All tables normalized to 3NF -- no data redundancy
- UUID primary keys for distributed system compatibility
- symptoms_json stored as JSONB for flexible symptom data per disease type
- Spatial indexing on area coordinates for fast geo-queries

DAY
07

Date: Jun-07

ER Diagram Design

Expected Deliverable

ER Diagram

Entity-Relationship Overview

All 10 database entities and their relationships are diagrammed below. Core entities, attributes, data types, and cardinalities are documented.

[USER] USERS	
[PK]	**user_id : UUID**
[FK]	area_id -> COMMUNITY_AREAS
-	name : VARCHAR(100)
-	email : VARCHAR(150)
-	password_hash : TEXT
-	role : ENUM
-	phone : VARCHAR(15)
-	created_at : TIMESTAMP

<-->

[AREA] COMMUNITY_AREAS	
[PK]	**area_id : UUID**
-	area_name : VARCHAR(100)
-	district : VARCHAR(100)
-	state : VARCHAR(80)
-	coordinates_json : JSON
-	population : INT
-	risk_level : ENUM
-	last_updated : TIMESTAMP

1 USER belongs to 1 COMMUNITY_AREA | 1 AREA has many USERS (1:N)

[SENSOR] IOT_SENSORS	
[PK]	**sensor_id : UUID**
[FK]	area_id -> COMMUNITY_AREAS
-	sensor_name : VARCHAR(100)
-	location_lat : DECIMAL
-	location_lng : DECIMAL
-	status : ENUM
-	installed_at : DATE
-	last_ping : TIMESTAMP

<-->

[READING] SENSOR_READINGS	
[PK]	**reading_id : UUID**
[FK]	sensor_id -> IOT_SENSORS
-	ph : FLOAT (0-14)
-	tds : FLOAT (ppm)
-	turbidity : FLOAT (NTU)
-	coliform : FLOAT (CFU/100ml)
-	temperature : FLOAT
-	recorded_at : TIMESTAMP

1 IOT_SENSOR has many SENSOR_READINGS (1:N -- time-series)

[SYMPTOM] SYMPTOM_REPORTS	
[PK]	**report_id : UUID**
[FK]	user_id -> USERS
[FK]	area_id -> COMMUNITY_AREAS
-	symptoms_json : JSONB
-	disease_category : VARCHAR
-	severity : ENUM
-	location_lat/lng : DECIMAL
-	reported_at : TIMESTAMP

<-->

[CASE] DISEASE_CASES	
[PK]	**case_id : UUID**
[FK]	area_id -> COMMUNITY_AREAS
[FK]	opened_by -> USERS
-	disease_type : VARCHAR(100)
-	affected_count : INT
-	status : ENUM
-	severity : ENUM
-	opened_at / closed_at : TIMESTAMP

SYMPTOM_REPORTS trigger DISEASE_CASES | 1 AREA has many of both (1:N)

[AI] OUTBREAK_PREDICTIONS	
[PK]	**prediction_id : UUID**
[FK]	area_id -> COMM_AREAS
-	risk_score : FLOAT (0-1)
-	risk_level : ENUM

<->

[ALERT] ALERTS	
[PK]	**alert_id : UUID**
[FK]	area_id -> COMM_AREAS
-	type : ENUM
-	message : TEXT
-	severity : ENUM

<->

[RECIP] ALERT_RECIPIENTS	
[PK]	**recipient_id : UUID**
[FK]	alert_id -> ALERTS
[FK]	user_id -> USERS
-	delivered_at : TIMESTAMP

-	model_version : VARCHAR
-	predicted_at : TIMESTAMP

-	sent_at : TIMESTAMP
-	channel : ENUM (push/SMS/email)
-	triggered_by : VARCHAR

-	read_status : BOOLEAN
-	channel_used : ENUM

OUTBREAK_PREDICTIONS -> ALERTS (triggers 1:N) | ALERTS -> ALERT_RECIPIENTS (1:N)

DAY
08

Date: Jun-08

Database Schema Creation

Expected Deliverable

SQL Schema

Schema Relationship Map

Relationship	Cardinality	Join Key
users -> community_areas	Many-to-1	users.area_id -> community_areas.area_id
community_areas -> iot_sensors	1-to-Many	iot_sensors.area_id -> community_areas.area_id
iot_sensors -> sensor_readings	1-to-Many (time-series)	sensor_readings.sensor_id -> iot_sensors.sensor_id
users -> symptom_reports	1-to-Many	symptom_reports.user_id -> users.user_id
community_areas -> symptom_reports	1-to-Many	symptom_reports.area_id -> community_areas.area_id
community_areas -> disease_cases	1-to-Many	disease_cases.area_id -> community_areas.area_id
community_areas -> outbreak_predictions	1-to-Many	outbreak_predictions.area_id -> community_areas.area_id
community_areas -> alerts	1-to-Many	alerts.area_id -> community_areas.area_id
alerts -> alert_recipients	1-to-Many	alert_recipients.alert_id -> alerts.alert_id
users -> alert_recipients	1-to-Many	alert_recipients.user_id -> users.user_id

Index Strategy

Table	Index Column(s)	Purpose
sensor_readings	sensor_id, recorded_at	Fast time-range queries for sensor trends
symptom_reports	area_id, reported_at, disease_category	Zone-based symptom cluster detection
disease_cases	area_id, status, opened_at	Active case filtering by zone
outbreak_predictions	area_id, predicted_at, risk_level	Latest risk score per zone lookup
users	email, area_id, role	Login and zone-based user queries
alerts	area_id, severity, sent_at	Recent critical alerts per zone

Key Constraints

- All primary keys: UUID with DEFAULT gen_random_uuid()
- Enum types defined as PostgreSQL ENUM or CHECK constraints

- sensor_readings: CASCADE DELETE when sensor deleted
- symptom_reports, disease_cases: RESTRICT DELETE on user/area
- alert_recipients: CASCADE DELETE when alert deleted
- InfluxDB used for sensor_readings in production (PostgreSQL for dev/testing)

DAY
09

Date: Jun-09

UI Wireframe Design

Expected Deliverable

Page Layouts

Wireframe Page List

Page #	Page Name	User Role	Key UI Elements
P-01	Landing / Home	All	Hero with disease stats, Login/Register CTA, Check your area quick search
P-02	Community Registration	Community Member	Name, email, phone, area (dropdown), password, emergency contact
P-03	Login Page	All	Email/password, role selector (member/officer/authority/admin), 2FA option
P-04	Community Member Dashboard	Member	My area risk badge, symptom report button, alert feed, water quality card
P-05	Symptom Report Form	Member	Symptoms checklist, severity slider, auto-GPS, disease category, submit
P-06	Area Risk Map	All	Heatmap of disease risk by zone (green/yellow/orange/red), filters
P-07	Water Quality Monitor	Water Authority	Live sensor readings (pH, TDS, turbidity), threshold alerts, sensor list
P-08	Health Officer Dashboard	Health Officer	Case list, symptom cluster map, outbreak predictions panel, alert trigger
P-09	Disease Case Management	Health Officer	Case details, affected count, update status, assign responders, notes
P-10	Outbreak Prediction Panel	Officer / Govt	AI risk scores per zone, confidence %, trend graph, historical comparison
P-11	Admin -- Sensor Registry	Admin	Add/edit/delete sensors, assign to zones, view ping status
P-12	Admin -- User Management	Admin	User table, role change, activate/deactivate, zone assignment
P-13	Analytics & Reports	Govt / Admin	Charts: cases/week, sensor trends, disease type breakdown, export PDF

UI Color Coding System

Color	Meaning	Used In
GREEN	Safe -- No risk detected	Zone maps, risk badges, sensor normal status
YELLOW	Caution -- Watch zone	Moderate sensor readings, rising symptom reports
ORANGE	Warning -- Elevated risk	High AI prediction score, multiple symptom clusters

RED	Critical -- Active Outbreak	Confirmed outbreak case, critical sensor contamination
BLUE	Informational	General alerts, water quality data, system status

DAY
10

Date: Jun-10

Login & Dashboard UI Design

Expected Deliverable
UI Screens

Login Screen Components

Component	Description
App Logo + Title	Health shield icon, Community Health Monitor in teal/blue
Email Input	Placeholder: Enter your email, real-time format validation
Password Input	Masked with show/hide toggle, strength indicator on register
Role Selector	Dropdown: Community Member / Health Officer / Water Authority / Admin
Login Button	Teal primary button, disabled until form valid, spinner on submit
Forgot Password	Link below button -> OTP-based reset flow
Register Link	New user? Register here -- shown for community members only
Error Display	Red inline error messages for invalid credentials

Community Member Dashboard

Component	Description
Header	App logo, area name, notification bell (badge count), user avatar, logout
Area Risk Card	Large colored badge: SAFE / CAUTION / WARNING / OUTBREAK with zone name
Report Symptom Button	Prominent teal button -- opens quick symptom report form
Active Alerts Feed	Scrollable list of recent health alerts for my area
Water Quality Card	Current pH, TDS, turbidity readings from nearest sensor
Nearby Safe Water	Map pins showing safe water points within 2km
Disease Tips	Seasonal health tips based on current risk level

Health Officer Dashboard

Component	Description
Statistics Bar	Total active cases, New symptom reports today, High-risk zones count, Alerts sent
Symptom Cluster Map	Heatmap of symptom reports in last 7 days by locality
Active Cases Panel	Sortable case list: disease type, area, severity, status, days open
Outbreak Prediction Widget	Top 3 zones by AI risk score with confidence percentage
Quick Alert Button	One-click send alert to selected zone
Sensor Status Row	Live status of all IoT sensors: online/offline/warning

DAY
11

Date: Jun-11

Navigation & Form Design

Expected Deliverable

UI Prototype

Navigation Access Matrix

Navigation Item	Member	Health Officer	Water Auth	Admin
Home / Dashboard	YES	YES	YES	YES
Report Symptoms	YES	NO	NO	NO
Area Risk Map	View only	Full Access	YES	YES
Water Quality Monitor	View only	YES	Full Access	YES
Disease Cases	NO	Full Access	NO	YES
Outbreak Predictions	View	Full Access	View	YES
Alert Center	Receive only	Send + Receive	Send + Receive	Full Access
Sensor Registry	NO	View only	Full Access	YES
User Management	Profile only	NO	NO	YES
Analytics & Reports	NO	YES	View only	YES

Key Forms Designed

Form	Fields	Validation
Symptom Report	Symptom checkboxes (diarrhea, vomiting, fever, jaundice), severity (1-5), notes, auto-GPS, photo upload	At least 1 symptom required, GPS mandatory
Disease Case Creation	Disease type (dropdown), affected count (INT), area, severity, description, assign officers	All fields required, count > 0
Sensor Registration	Sensor name, area (dropdown), lat/lng, install date, reading interval	Lat/Lng must be valid coordinates
Manual Alert Send	Zone(s) multiselect, alert type, message, channel (push/SMS/email), severity	Message max 200 chars, zone required
Water Threshold Config	Parameter (pH/TDS/turbidity/coliform), min/max safe values, alert delay	Min must be < Max, numeric only

Prototype Flow

- Member flow: Landing -> Register -> Dashboard -> Report Symptom -> View Alert
- Health Officer flow: Login -> Dashboard -> View Cluster Map -> Open Case -> Trigger Alert
- Water Authority flow: Login -> Sensor Dashboard -> View Readings -> Log Contamination -> Alert
- Admin flow: Login -> Admin Panel -> Register Sensor -> Manage Users -> View Analytics

DAY
12

Date: Jun-12

Design Review

Expected Deliverable

Design Approval

Design Review Checklist

Review Area	Status	Comments
Project Title	APPROVED	Relevant and impactful -- public health domain
Problem Statement	APPROVED	Well-scoped, data-driven gap identified
Objectives (8)	APPROVED	All SMART, feasible within 30 days
User Roles (6)	APPROVED	All stakeholders covered
Module List (10)	APPROVED	Comprehensive, no gaps
Use Cases	APPROVED	All actors and flows documented
CRUD API Matrix	APPROVED	REST conventions followed correctly
Database Tables (10)	APPROVED	3NF normalized, good use of JSONB for symptoms
ER Diagram	APPROVED	All entities and relationships correctly mapped
SQL Schema	APPROVED	Indexes and constraints properly defined
UI Wireframes (13 pages)	APPROVED	All screens covered, color system logical
UI Screens	APPROVED	Dashboard components are clear and functional
Navigation Matrix	APPROVED	Role-based access well-designed
UI Prototype	MINOR FIX	Symptom report form -- add photo upload progress indicator
Overall Design	APPROVED	Ready for development phase

Faculty Review Feedback

Faculty Comments:

1. Excellent project selection -- IoT + AI for public health is highly relevant and innovative.
2. Ensure AI model is validated with real water quality and disease datasets -- cite sources in report.
3. InfluxDB integration for sensor time-series is a good architectural choice -- document the setup.
4. The geo-spatial heatmap will be the visual highlight -- prioritize it for demo day.
5. **Overall: Design phase excellently executed. APPROVED for development!**

DAY
13

Date: Jun-13

Frontend Environment Setup

Expected Deliverable

React Project Setup

Frontend Tech Stack

Tool / Package	Version	Purpose
Node.js	v20.x LTS	JavaScript runtime
npm	v10.x	Package manager
React.js	v18.2	Frontend UI framework
Vite	v5.x	Fast build tool
Tailwind CSS	v3.4	Utility-first CSS framework
React Router DOM	v6.x	Client-side routing
Axios	v1.6	HTTP client for REST API calls
React Leaflet	v4.x	Interactive geo-spatial maps
Chart.js / Recharts	v3.x	Sensor trend charts, analytics graphs
Socket.IO Client	v4.x	Real-time WebSocket for live sensor + alerts
React Toastify	v10.x	Toast notifications for alerts
JWT Decode	v4.x	JWT token parsing on client side
ESLint + Prettier	Latest	Code quality and formatting

Project Folder Structure

```
community-health-frontend/
|-- src/
|   |-- components/   # Reusable: Map, SensorCard, AlertBanner, RiskBadge
|   |-- pages/        # Dashboard, SymptomReport, SensorMonitor, CaseManager
|   |-- services/     # api.js (Axios), alertService.js, sensorService.js
|   |-- context/      # AuthContext, AlertContext, ZoneContext
|   |-- hooks/        # useSensorData, useOutbreakRisk, useGeoLocation
|   |-- utils/        # riskCalculator.js, dateFormatter.js, mapHelpers.js
|   `-- App.jsx       # Root with protected routes by role
|-- public/           # favicon, index.html, manifest.json
`-- package.json      # Dependencies manifest
```

Environment Variables (.env)

- VITE_API_BASE_URL=http://localhost:8080/api
- VITE_MAPS_API_KEY=[Google Maps API Key]
- VITE_WS_URL=ws://localhost:8080/ws (real-time sensor + alerts)
- VITE_FIREBASE_KEY=[Firebase Push Notification Key]
- VITE_INFLUX_URL=http://localhost:8086 (sensor time-series)

Days 1-13 Summary

Day	Date	Planned Activity	Deliverable
01	Jun-01	Project Title Finalization	Approved Project Title
02	Jun-02	Requirement Gathering	Problem Statement
03	Jun-03	Objective Definition	Project Objectives
04	Jun-04	User & Module Identification	Module List
05	Jun-05	Use Case Diagram Preparation	CRUD APIs
06	Jun-06	Database Requirement Analysis	Table List
07	Jun-07	ER Diagram Design	ER Diagram
08	Jun-08	Database Schema Creation	SQL Schema
09	Jun-09	UI Wireframe Design	Page Layouts
10	Jun-10	Login & Dashboard UI Design	UI Screens
11	Jun-11	Navigation & Form Design	UI Prototype
12	Jun-12	Design Review	Design Approval
13	Jun-13	Frontend Environment Setup	React Project Setup